

**INSTITUTO
FEDERAL**
Santa Catarina

Conceitos Básicos de Kotlin

Programação para Aplicativos Móveis

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br



cores

tom escuro	#45474B	outer space
acento escuro	#D45113	syracuse red-orange
cor principal	#006674	caribbean current
acento claro	#FFBA08	selective yellow
tom claro	#F5F7F8	anti-flash white

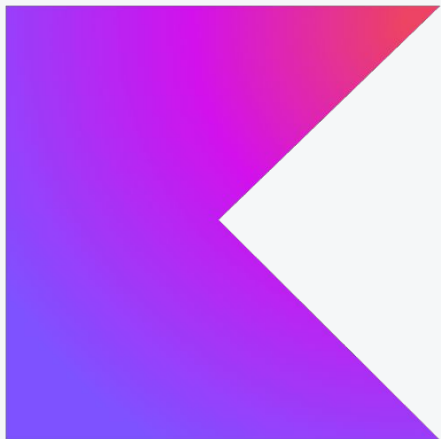
fontes

Fira Sans Extra Condensed

Ubuntu

Roboto Mono

Kotlin



breve histórico

- 2016 lançado oficialmente pela JetBrains
- 2017 torna-se a linguagem oficial para desenvolvimento Android
- multiplataforma (JVM, nativa, JavaScript, WebAssembly)
- concisa e segura contra nulidade
- suporte a programação funcional e orientada a objetos

Conceitos Básicos de Kotlin

sintaxe básica

- não é necessário o ponto e vírgula (;) final
- o compilador infere o tipo de dado das variáveis
 - é possível declarar tipos explicitamente
- convenção:
 - nomes de variáveis e funções: camelCase
 - nomes de classes: PascalCase
- arquivos Kotlin devem ter a extensão .kt
- os arquivos podem conter múltiplas declarações de funções e classes
- os arquivos devem ser organizados em pacotes
 - é recomendável que a estrutura de diretórios reflita a hierarquia dos pacotes
 - Java e Kotlin usam a mesma estrutura de diretórios
 - src/main/java
 - src/main/kotlin
 - arquivos de teste
 - src/test/java
 - src/test/kotlin

Conceitos Básicos de Kotlin

estrutura de um programa

- não é necessário declarar uma classe para o programa
- os parâmetros são opcionais na função main

exemplos

```
// Java
package ads.mov;

public class Main {
    public static void main(String[] args) {
        System.out.println("Olá, mundo!");
    }
}
```

```
// Kotlin
package ads.mov;

fun main() {
    println("Olá mundo!")
}
```

Conceitos Básicos de Kotlin

estrutura de um programa

- não é necessário declarar uma classe para o programa
- os parâmetros são opcionais na função main
- o arquivo .kt pode ter qualquer nome, sendo comum usar Main.kt
- não existe o conceito de static em Kotlin

exemplos

```
package ads.mov

fun olaMundo() {
    println("Olá mundo!")
}

fun main() {
    olaMundo()
}
```

Conceitos Básicos de Kotlin

variáveis

- declaração feita com as palavras-chave:
 - `val`, para variáveis imutáveis
 - `var`, para variáveis mutáveis
- variáveis imutáveis
 - previnem modificações não intencionais, tornando o código mais seguro e previsível
 - facilitam a concorrência, eliminando a necessidade de sincronização para acesso a dados
 - melhoram a performance, permitindo otimizações pelo compilador

exemplos

```
// Declaração de variáveis com tipo explícito
var nome: String // tipo String, mutável
nome = "João" // atribuição de valor
```

```
// Declaração de variáveis com tipo explícito e
// inicialização
var idade: Int = 30
// tipo Int, mutável e inicializado com 30
```

```
// Declaração de variáveis com tipo inferido
val valor = 25.0 // tipo inferido como Int, imutável
var ativo = true // tipo inferido como Boolean, mutável
```

```
idade = 20 // atribuição de novo valor
// valor = 3.0 // erro: não é possível atribuir valor a
// uma variável imutável
```

Conceitos Básicos de Kotlin

tipos primitivos

tipo	bytes	descrição	exemplo
Int	4	inteiro	<code>var i: Int = 10</code>
Long	8	inteiro longo	<code>var l: Long = 100000L</code>
Short	16	inteiro curto	<code>var s: Short = 1000</code>
Float	4	ponto flutuante	<code>var f: Float = 3.14f</code>
Double	8	ponto flutuante longo	<code>var d: Double = 3.14159</code>
Char	2	caractere	<code>var c: Char = 'A'</code>
Byte	1	byte	<code>var y: Byte = 127</code>
Bool	1 bit	lógico	<code>var b: Boolean = true</code>

Conceitos Básicos de Kotlin

operadores

tipo	operadores	descrição
aritméticos	+, -, *, /, %	soma, subtração, multiplicação, divisão, módulo
comparação	==, !=, <, >, <=, >=	igualdade, diferença, maior/menor
lógicos	&&, , !	E, OU, NÃO lógico
atribuição	=, +=, -=, *=, /=, %=	atribuição e operações compostas
inc/dec	++, --	incremento/decremento
intervalo	in, !in	pertence ou não a um intervalo
tipo	is, !is	verifica tipo
elvis	?:	valor padrão se nulo
chamada segura	?.	acesso seguro a membros de objeto nulo

Conceitos Básicos de Kotlin

segurança contra nulidade

- Kotlin é projetada para evitar erros de nulidade
- variáveis que podem ser nulas devem ser declaradas com o sufixo ?
 - obriga o programador a lidar explicitamente com valores nulos

exemplos

```
// Declaração de variável que permite nulo  
var nome: String?
```

```
// Declaração de variável que não permite nulo  
var cidade: String
```

```
nome = null // Válido, pois permite nulo  
cidade = null // ERRO, pois não permite guardar nulo
```

Conceitos Básicos de Kotlin

segurança contra nulidade

- operador de chamada segura ?.
 - permite acessar membros de um objeto apenas se ele não for nulo
 - retornando null caso contrário
- operador Elvis ?:
 - fornece um valor padrão quando uma expressão resulta em null
 - tornando o código mais robusto

exemplos

```
var nome: String? = null
```

```
// se 'nome' for nulo, retorna o valor à direita  
println(nome ?: "Nome não informado")  
// saída: Nome não informado
```

```
nome = "Maria"  
// só acessa 'length' se 'nome' não for nulo  
println(nome?.length) // Saída: 5
```

```
// retorna o tamanho ou zero se 'nome' for nulo  
val tamanho = nome?.length ?: 0  
println("Tamanho do nome: $tamanho")
```

Conceitos Básicos de Kotlin

leitura de dados pelo teclado

- `readln`
 - lê uma linha do teclado e retorna uma `String` não nula
- `readlnOrNull`
 - lê uma linha do teclado e retorna uma `String` ou nulo

exemplos

```
fun main() {  
  
    print("Digite seu nome: ")  
    // Operador Elvis (?:) fornece um valor padrão caso a  
    // entrada seja nula  
    val nome = readlnOrNull() ?: ""  
  
    print("Digite sua idade: ")  
    // toIntOrNull() converte a String para Int,  
    // retornando nulo se a conversão falhar  
    val idade = readlnOrNull()?.toIntOrNull() ?: 0  
}
```

Conceitos Básicos de Kotlin

números

- são objetos em Kotlin
- têm funções que podem ser invocadas

exemplos

```
package ads.mov

fun main() {
    val numero1 = 10
    val numero2 = 3

    // Operações aritméticas
    println("Soma: ${numero1.plus(numero2)}")
    println("Multiplicação: ${numero1.times(numero2)}")
    println("Divisão: ${numero1.div(numero2)}")
    println("Módulo: ${numero1.rem(numero2)}")

    // Funções de comparação
    println("É maior que? ${numero1 > numero2}")
    println("É menor que? ${numero1 < numero2}")

    // Converter um Int para String
    val numeroComoString = numero1.toString()
}
```

Conceitos Básicos de Kotlin

strings

- podem ser declaradas com três aspas (") para strings multilinha
 - permitem quebras de linha e indentação sem a necessidade de caracteres especiais
 - podem ser usadas para criar *templates* de texto
 - no início de uma linha, o caractere | pode ser usado para indicar a margem
- podem conter expressões interpoladas usando o símbolo \$ ou \${expressão} para avaliar expressões dentro da string

exemplos

```
val nome = "João"
val idade = 30
val mensagem = "Olá, meu nome é $nome e tenho $idade anos."
val variasLinhas = """Olá, meu nome é $nome
|E tenho $idade anos.
|Essa é uma string multilinha.""".trimMargin()
println(mensagem)
println(variasLinhas)

val str1 = "Olá"
val str2 = "Mundo"
val str3 = "$str1 $str2" // Concatenação
println("Tamanho da string: ${str3.length}")
println("Primeiro caractere: ${str3[0]}")
println("Substituição: ${str3.replace("Mundo", "Kotlin")}")
```

Conceitos Básicos de Kotlin

vetores (arranjos)

- a classe `Array` é usada para criar vetores com tamanho fixo
- em Kotlin é preferível usar coleções como `List`, `Map` ou `Set` para manipulação de dados

exemplos

```
val numeros = arrayOf(1, 2, 3, 4, 5)

println("Tamanho do vetor: ${numeros.size}")
println("Primeiro elemento: ${numeros[0]}")
println("Último elemento: ${numeros[numeros.size - 1]}")
println("Todos os elementos: ${numeros.joinToString(", ")}")
println("Elementos pares: ${numeros.filter { it % 2 == 0 }}")

// cria vetor de inteiros com tamanho 5, sem
// inicialização
var idade = IntArray(5)
// cria vetor com tamanho 10, inicializado com zeros
var outro = Array(10) { 0 }
// cria vetor de strings com tamanho 3, inicializado com
// strings vazias
var nomes = Array(3) { "" }
```

Conceitos Básicos de Kotlin

coleções

- **listas** coleções ordenadas de elementos, que podem conter duplicatas
 - **listOf()** cria uma lista imutável
 - **mutableListOf()** cria uma lista mutável
- **mapas** coleções de pares chave-valor, onde cada chave é única
 - **mapOf()** cria um mapa imutável
 - **mutableMapOf()** cria um mapa mutável
- **conjuntos** coleções de elementos únicos, sem ordem específica
 - **setOf()** cria um conjunto imutável
 - **mutableSetOf()** cria um conjunto mutável

exemplos

```
val num = listOf(1, 2, 3, 4, 5)
val nMutavel: MutableList<Int> = mutableListOf(1, 2, 3)
println("Lista de números: $num")
```

```
val nomes = mutableListOf("João", "Maria", "Pedro")
nomes.add("Ana") // Adiciona um elemento
println("Lista de nomes: $nomes")
```

```
val mapa = mapOf("a" to 1, "b" to 2, "c" to 3)
println("Mapa: $mapa")
```

```
val mapaMutavel = mutableMapOf("x" to 10, "y" to 20)
mapaMutavel["z"] = 30 // Adiciona um novo par chave-valor
println("Mapa mutável: $mapaMutavel")
```


Conceitos Básicos de Kotlin

tipos em coleções e vetores

- é possível criar coleções com tipos misturados
 - é recomendável evitar isso para manter a clareza do código
- para coleções com tipos misturados, é comum usar o tipo Any
 - Any é a superclasse de todos os tipos

exemplos

```
val misturado = mutableListOf(1, "dois", 3.0, true)
println("Lista misturada: $misturado")
```

```
val sturadoComAny: MutableList<Any> = mutableListOf(1,
"dois", 3.0, true)
```

```
val misturado2 = mutableListOf<Any>(1, "dois", 3.0, true)
```

```
val arranjo = arrayOf<Any>("Juca", 10)
```

Conceitos Básicos de Kotlin

condicionais

```
val idade = 17
if (idade < 16) {
    println("Não pode votar no Brasil")
} else if (idade in 16..17) {
    println("Pode votar no Brasil")
} else {
    println("Pode votar e dirigir no Brasil")
}

// equivalente ao operador ternário em Java
val numero = if (10 % 2 == 0) "par" else "ímpar"

val numero = 2
when (numero) {
    1 -> println("Número é 1")
    2 -> println("Número é 2")
    else -> println("Número desconhecido")
}
```

```
when (numero) {
    1, 2 -> println("Número é 1 ou 2")
    in 3..5 -> println("Número está entre 3 e 5")
    else -> println("Número desconhecido")
}

// como expressão
val resultado = when (numero) {
    1 -> "Um"
    2 -> "Dois"
    else -> "Outro"
}
```

Conceitos Básicos de Kotlin

repetição

```
for (i in 1..5) {  
    println("Número: $i")  
}
```

```
for (i in 1..10 step 2) { // com passo de 2  
    println("Número: $i")  
}
```

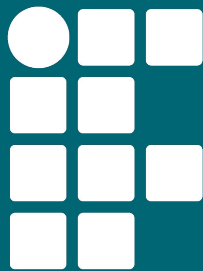
```
val numeros = listOf(1, 2, 3, 4, 5)  
for (numero in numeros) {  
    println("Número: $numero")  
}
```

```
for ((indice, numero) in numeros.withIndex()) {  
    println("Número na posição $indice: $numero")  
}
```

```
val mapa = mapOf("a" to 1, "b" to 2, "c" to 3)  
for ((chave, valor) in mapa) {  
    println("Chave: $chave, Valor: $valor")  
}
```

```
var j = 0  
while (j < 5) {  
    println("Número: $j")  
    j++  
}
```

```
var k = 0  
do {  
    println("Número: $k")  
    k++  
} while (k < 5)
```



**INSTITUTO
FEDERAL**
Santa Catarina

Conceitos Básicos de Kotlin

Programação para Aplicativos Móveis

Análise e Desenvolvimento de Sistemas

Prof. Adriano Lima

adriano.lima@ifsc.edu.br

